Bare Metal desde Cero

Volumen 1: Arquitectura, Lenguaje C y Control de Registros en el STM32G474RE

1. ¿Qué significa "Bare Metal"?

Imagina que compras una computadora. Para usarla, necesitas Windows o Linux (un Sistema Operativo). El SO actúa como intermediario: tú haces clic, el SO procesa, y luego el SO le dice al hardware qué hacer.

En el mundo de los microcontroladores (como tu STM32G474RE), programar **Bare Metal** significa que **no hay intermediarios**. No hay sistema operativo. Escribes código en lenguaje C que habla directamente con los transistores y registros físicos de silicio dentro del chip. Tú tienes el control absoluto de cada ciclo de reloj y cada voltaje que entra o sale de la placa.

> **Tu Herramienta: STM32G474RE (Nucleo-G474RE)**
>
> El "cerebro" de tu placa es un procesador ARM Cortex-M4 de 32 bits corriendo a 170 MHz. Es increíblemente rápido para tareas industriales, matemáticas avanzadas y control de motores eléctricos. Al tener una placa blanca (Nucleo), la parte superior (donde va el USB) es un programador físico llamado ST-LINK que te permitirá inyectar tu código al chip.

2. Anatomía Interna Básica

Para programar hardware, debes saber qué hay dentro de esa pastilla de plástico negro:

Componente	Función en la vida real	¿Qué hace en Bare Metal?
CPU (ARM Cortex-M4)	El Director de Orquesta	Lee tu código línea por línea y ejecuta cálculos matemáticos o decisiones (if/else).
Flash Memory (512 KB)	El Disco Duro	Aquí se guarda tu código compilado. Si desconectas la placa, el código no se borra.
SRAM (128 KB)	La Memoria a Corto Plazo	Guarda las variables y cálculos temporales mientras la placa tiene energía.
Periféricos	Los Brazos y Ojos	Hardware dedicado (Timers, Puertos GPIO, Conversores ADC) que interactúa con el mundo exterior.

3. El Lenguaje C: Tipos de Datos Estrictos

Si no sabes C, la regla de oro es: **cada dato necesita una caja (variable) de un tamaño exacto**. En PC usamos `int` para todo. En Bare Metal, la memoria es oro y los registros físicos tienen tamaños exactos (8, 16 o 32 bits). Por eso, siempre incluimos la librería `<stdint.h>`, que nos da tipos de datos con tamaños bloqueados.

Tipo en C	Significado	Tamaño físico	Rango de Valores	Uso típico en Bare Metal
uint8_t	Entero Sin Signo (Unsigned)	8 bits (1 Byte)	0 a 255	Controlar un puerto pequeño, enviar letras por puerto serial.
uint16_t	Entero Sin Signo	16 bits (2 Bytes)	0 a 65,535	Leer sensores analógicos (ADC), usar Timers básicos.
uint32_t	Entero Sin Signo	32 bits (4 Bytes)	0 a 4,294,967,295	**Configurar los registros principales del STM32.**
int8_t / int32_t	Enteros Con Signo (Signed)	8 o 32 bits	Incluyen negativos	Matemáticas, control PID, sensores de temperatura (bajo cero).

Ejemplo de código básico:

```c
#include <stdint.h>

void main(void) {
    // Creamos una variable de 8 bits para guardar un porcentaje
    uint8_t velocidad_motor = 50;

    // Creamos una variable de 32 bits para una configuración gigante
    uint32_t configuracion_sistema = 0xFFFFFFFF; // Lleno de unos

    // Bucle infinito: Un microcontrolador NUNCA debe terminar su programa
    while (1) {
        if (velocidad_motor < 100) {
            velocidad_motor = velocidad_motor + 1;
        }
    }
}
```

> **Hexadecimal (0x)**
> En el código verás mucho números que empiezan con 0x (ej. 0x1A). Es la forma hexadecimal. A los ingenieros les encanta porque es muy fácil convertir de hexadecimal a bits binarios en la cabeza, mucho más fácil que usar números decimales normales.

4. Operaciones Bit a Bit (El Idioma del Hardware)

Los registros dentro del STM32 son simplemente filas de 32 interruptores (bits). Cada interruptor controla algo físico (ej. encender un LED o activar un reloj). El problema es que en C no puedes decirle "cambia el bit número 5". Tienes que enviar un número de 32 bits completo.

Para encender o apagar un solo interruptor sin dañar accidentalmente la configuración de los otros 31, usamos las operaciones lógicas **Bitwise**. Esto es el 90% de la programación Bare Metal.

4.1 Encender un Bit (Operación OR `|`)

Usamos la compuerta lógica OR. Si hacemos OR con un 1, el bit se enciende forzosamente. Si hacemos OR con un 0, el bit original se queda intacto.

```
// Imaginemos que REGISTRO es una variable de hardware de 8 bits
// Actualmente vale: 0b10000000
uint8_t REGISTRO = 128;

// Queremos ENCENDER el bit en la posición 1 (el segundo de derecha a izq)
// Creamos una "Máscara" moviendo un '1' a la posición deseada usando <<
// Máscara: (1 << 1) es igual a 0b00000010

REGISTRO = REGISTRO | (1 << 1);  // Forma larga
REGISTRO |= (1 << 1);            // Forma abreviada y profesional

// Resultado físico: El bit 1 se encendió, el bit 7 sigue encendido.
// Nuevo valor: 0b10000010
```

4.2 Apagar un Bit (Operación AND `&` y NOT `~`)

Usamos la compuerta AND. Hacer AND con un 0 apaga el bit, hacer AND con un 1 lo deja intacto. Para esto, creamos una máscara con el bit encendido, y luego la invertimos (NOT ~) para que ese bit sea el único cero.

```
// Queremos APAGAR el bit en la posición 7
// 1. Máscara: (1 << 7)            -> 0b10000000
// 2. Invertida: ~(1 << 7)         -> 0b01111111

REGISTRO &= ~(1 << 7);  // Aplica la máscara invertida con AND

// Resultado físico: El bit 7 se apagó, el bit 1 sigue encendido.
// Nuevo valor: 0b00000010
```

4.3 Invertir un Bit (Operación XOR ^)

La compuerta XOR es mágica. Si aplicas XOR con un 1, el bit original se invierte (si era 0 pasa a 1, si era 1 pasa a 0). Es la operación más usada para hacer parpadear un LED o crear una onda cuadrada.

```
// Queremos ALTERNAR el estado del bit 1 (hacer parpadear un LED)
REGISTRO ^= (1 << 1);

// Si el código pasa por aquí repetidas veces en un bucle:
// Vez 1: 0b00000000 -> 0b00000010 (LED Enciende)
// Vez 2: 0b00000010 -> 0b00000000 (LED Apaga)
// Vez 3: 0b00000000 -> 0b00000010 (LED Enciende)
```

4.4 Leer / Comprobar un Bit (Operación AND &)

Cuando quieres saber si un botón físico está presionado, tienes que leer el registro. Usas una máscara AND para aislar (filtrar) el bit que te interesa, ocultando todos los demás bits del puerto.

```
// Queremos saber si el bit 3 está encendido
// REGISTRO actual: 0b11001000 (Vemos que el bit 3 sí es 1)

if (REGISTRO & (1 << 3)) {
    // Si el resultado no es cero, significa que el bit estaba en 1.
    // Aquí pones el código: "Encender motor porque se presionó el botón"
} else {
    // El bit estaba en 0.
    // El botón NO está presionado.
}
```

> **Resumen de Oro para Bare Metal**
>
> - Para **Encender (Set)** usa: `|=`
> - Para **Apagar (Clear)** usa: `&= ~`
> - Para **Alternar (Toggle)** usa: `^=`
> - Para **Leer (Check)** usa: `if (REG & (1 << bit))`
>
> Si memorizas esto, ya puedes controlar hardware.

5. Punteros y Direcciones de Memoria

La memoria del STM32 es como una gran calle llena de buzones físicos. Cada buzón puede guardar 32 bits (4 bytes) de información, y cada buzón tiene una **Dirección Postal** única escrita en hexadecimal (ej. 0x40020000).

Un puntero en C no es una variable normal que guarda un número; es una variable que **guarda direcciones postales**. Sirven para decirle a la CPU: "Oye, ve a la dirección 0x40020000 física del silicio y cambia los interruptores ahí".

> **La Sintaxis de los Punteros**
>
> - * (Asterisco): Se usa para **crear** un puntero, y también para **ver qué hay dentro** del buzón al que apunta (desreferenciar).
> - & (Ampersand): Se usa para descubrir la dirección postal física de una variable existente.

Forzando la CPU a escribir en hardware:

```
// 1. Definimos la dirección física (buscada en el manual del STM32)
// Supongamos que el registro para encender un pin está en 0x48000014

// 2. Creamos un Puntero y le decimos a C que ahí hay un número de 32 bits
uint32_t *puntero_registro = (uint32_t *)0x48000014;

// 3. Vamos a la dirección (usando *) y escribimos físicamente un 1
*puntero_registro |= (1 << 5); // ¡El LED se acaba de encender en la vida real!
```

Este bloque de código arriba es, literalmente, la esencia más pura del Bare Metal. Escribir directamente en el mapa de memoria.

6. La Palabra Mágica: `volatile`

Los compiladores modernos (como el GCC que usa tu VS Code) son muy inteligentes. Si el compilador ve un bucle de código que lee una variable pero nunca la modifica, el compilador "optimiza" el programa asumiendo que esa variable siempre vale lo mismo, y deja de revisarla para ahorrar tiempo de procesamiento.

En hardware, esto es catastrófico. Un registro físico puede cambiar sin que el software lo haga (por ejemplo, alguien apretó un botón en el mundo real, o un sensor de temperatura detectó calor). Si el compilador ignoró ese registro, el botón nunca será detectado.

```
// INCORRECTO para hardware:
uint32_t *registro_boton = (uint32_t *)0x48000010;

// CORRECTO: Obliga a la CPU a leer el hardware REAL en cada milisegundo
volatile uint32_t *registro_boton = (volatile uint32_t *)0x48000010;
```

7. El Mapa de Memoria del STM32G4

En la arquitectura de ARM Cortex-M, todo (RAM, Flash, Pines, Timers) comparte el mismo "barrio" de direcciones. El chip asigna cuadras específicas a cada cosa. Esta es la distribución base:

Dirección Base	Sección	¿Qué hay físicamente ahí?
0x00000000	Memoria Flash	Donde se graba tu código C compilado.
0x20000000	Memoria SRAM	Tus variables temporales y punteros viven aquí.
0x40000000	Periféricos (APB/AHB)	**¡La zona de juegos!** Aquí están los puertos físicos, Timers, ADC, UART, etc. Escribiremos en esta zona el 99% del tiempo.

> **Los Buses de Hardware**
> Internamente, el microcontrolador tiene "autopistas" de cables para mover datos. Los periféricos lentos están en los buses **APB** (Advanced Peripheral Bus), y los más rápidos en los **AHB** (Advanced High-performance Bus).

8. CMSIS y Estructuras (Mapeo de Hardware)

Escribir direcciones hexadecimales completas como `0x48000014` en cada línea de código es un suicidio de ingeniería: te vas a equivocar, y el código será imposible de leer por otra persona.

La empresa ARM y STMicroelectronics crearon el estándar **CMSIS**. Proporcionan un archivo gigante (ej. `stm32g474xx.h`) que usa estructuras de C (`struct`) para mapear todas las direcciones. Cada puerto GPIO (A, B, C...) tiene múltiples registros, y todos están ordenados a la misma distancia en la memoria.

¿Cómo lo hace la librería por detrás?

```
// 1. Crean una estructura que agrupa los registros físicos en el orden real del chip
typedef struct {
  volatile uint32_t MODER;   // Registro para Modo (Entrada o Salida)
  volatile uint32_t OTYPER;  // Registro para Tipo de salida
  volatile uint32_t OSPEEDR; // Registro para Velocidad
  volatile uint32_t PUPDR;   // Registro para Resistencias internas
  volatile uint32_t IDR;     // Registro para LEER pines (Entrada)
  volatile uint32_t ODR;     // Registro para ESCRIBIR pines (Salida)
} GPIO_TypeDef;

// 2. Definen la dirección base donde empieza el Puerto A físicamente
#define GPIOA_BASE   (0x48000000)

// 3. Crean un Puntero (llamado GPIOA) que apunta a esa dirección con el molde del
struct
#define GPIOA        ((GPIO_TypeDef *) GPIOA_BASE)

// ¡AHORA PODEMOS PROGRAMAR COMO HUMANOS!
// Para escribir en el registro ODR del Puerto A, simplemente usamos la flecha '->'
main() {
    GPIOA->ODR |= (1 << 5); // Encender el bit 5 del puerto A
}
```

A partir de ahora, nunca escribiremos números hexadecimales directos; usaremos siempre las estructuras que trae el archivo `stm32g474xx.h`.

9. El Primer Paso Obligatorio: RCC (El Reloj)

En Arduino, escribes `digitalWrite` y todo funciona. En STM32 Bare Metal, si intentas encender un puerto, la placa simplemente ignorará tu código. ¿Por qué?

Porque el STM32 es de ultra-bajo consumo energético. Por defecto, cuando lo conectas a la energía, **el 95% del chip está desconectado internamente del reloj central** (está en estado vegetal para ahorrar batería). Antes de usar cualquier periférico (puertos, timers, conversor), debemos ir al módulo **RCC (Reset and Clock Control)** y enviarle energía a ese sector específico.

Registro Principal: `RCC->AHB2ENR` (AHB2 Peripheral Clock Enable Register)

Este registro controla la energía de los puertos físicos GPIO. Cada bit enciende un puerto distinto.

- Bit 0: Habilita el reloj para el Puerto A (GPIOA)
- Bit 1: Habilita el reloj para el Puerto B (GPIOB)
- Bit 2: Habilita el reloj para el Puerto C (GPIOC)

Ejemplo: Encender el Puerto A y el Puerto C

El LED integrado en tu placa Nucleo suele estar en el Pin PA5 (Puerto A, Pin 5). El Botón de usuario (User Button azul) suele estar en el Pin PC13 (Puerto C, Pin 13). Necesitamos encender ambos puertos.

```
#include "stm32g474xx.h"

void System_Clock_Init(void) {
    // Vamos a la estructura RCC, registro AHB2ENR
    // Encendemos el bit 0 (GPIOA) y el bit 2 (GPIOC)

    RCC->AHB2ENR |= (1 << 0); // Set Bit 0 para GPIOA
    RCC->AHB2ENR |= (1 << 2); // Set Bit 2 para GPIOC

    // Alternativa más profesional y directa:
    // RCC->AHB2ENR |= (RCC_AHB2ENR_GPIOAEN | RCC_AHB2ENR_GPIOCEN);
}
```

10. GPIO (Puertos Físicos) - Configurar Dirección

Ya que el Puerto A tiene energía, la CPU necesita saber qué vas a hacer con él. ¿Vas a enviar electricidad hacia afuera para encender un LED? ¿O vas a leer la electricidad que viene de afuera desde un botón? A esto se le llama configurar la dirección del puerto.

Registro: `GPIOA->MODER` (GPIO port Mode Register)

¡Atención aquí! A diferencia del registro anterior donde 1 bit controlaba 1 puerto, en el MODER **se usan 2 bits para configurar 1 pin**. Como el puerto A tiene 16 pines físicos, 2 bits × 16 pines = ocupa los 32 bits completos del registro.

- `00` : Modo de Entrada (Input mode) - Por defecto (Reset state)
- `01` : Modo de Salida (Output mode)
- `10` : Modo Alterno (Para UART, Timers, PWM, SPI)
- `11` : Modo Analógico (Para leer voltajes variables con el ADC)

Configurar el Pin PA5 (LED) como Salida

El Pin 5 es controlado por los bits [11:10]. Queremos que valgan `01`.

```
void GPIO_Init(void) {
    // PASO 1: Limpiar los bits 10 y 11 con AND (Ponerlos a 00)
    // (El bit 11 y 10 equivalen a 3 desplazado 10 veces, es decir 11 binario)
    GPIOA->MODER &= ~(3 << 10);  // Limpia ambos bits

    // PASO 2: Escribir un 01 en esa posición (Modo Salida)
    GPIOA->MODER |= (1 << 10);   // Enciende solo el bit 10

    // Opcional: El botón PC13 arranca en 00 (Entrada) por defecto al resetear.
    // Por lo que no hace falta configurar el GPIOC->MODER para leer el botón.
}
```

11. GPIO - Encender Apagar (Output Data)

Una vez configurado el pin como Salida (Output), controlar el voltaje es extremadamente sencillo. Simplemente escribimos un 1 para sacar 3.3V (Voltaje Alto - HIGH) o un 0 para sacar 0V (Voltaje Bajo - GND / LOW).

Registro: `GPIOA->ODR` (Output Data Register)

Este es el registro de escritura directa. Sus primeros 16 bits corresponden a los 16 pines físicos del Puerto A. Aquí 1 bit = 1 pin.

- Escribir `1` en el bit 5 : El pin PA5 saca voltaje (LED Enciende).
- Escribir `0` en el bit 5 : El pin PA5 cae a GND (LED Apaga).

Código: Hacer Parpadear un LED

Crearemos una función simple para gastar ciclos de reloj de la CPU y generar un retraso (delay) artificial muy tosco.

```
// Función simple de retardo gastando tiempo de la CPU
void delay_basico(volatile uint32_t count) {
    while(count--) {
        // No hacer nada, solo decrementar hasta cero
    }
}

int main(void) {
    System_Clock_Init(); // Encendemos la energía (Página 8)
    GPIO_Init();         // Configuramos PA5 como salida (Página 9)

    while(1) {
        // Usamos XOR (^) para invertir el estado del bit 5
        GPIOA->ODR ^= (1 << 5);

        // Esperamos un tiempo (Aprox medio segundo a 170MHz, dependerá)
        delay_basico(1000000);
    }
}
```

12. GPIO - Leer Sensores y Botones (Input Data)

Cuando configuramos un pin físico como Entrada (Input), el microcontrolador pasa a estar "escuchando" qué voltaje llega de afuera. Si llega un voltaje cercano a 3.3V, lo interpreta como un 1 lógico. Si llega GND (0V), lo interpreta como un 0 lógico.

> **Registro: `GPIOC->IDR` (Input Data Register)**
> A diferencia del ODR, el IDR es de **Solo Lectura (Read Only)**. No podemos escribir en él. El hardware físico del STM32 lo actualiza constantemente reflejando la realidad eléctrica de los pines. Aplicaremos una máscara AND para leer el pin específico.

Código: Controlar LED con el Botón

El botón azul de la placa Nucleo (PC13) suele estar conectado de forma invertida (Pull-Up externo). Es decir, siempre envía un 1 a la CPU, y cuando lo presionas, envía un 0.

```
int main(void) {
    System_Clock_Init();
    GPIO_Init(); // PA5 Salida (LED), PC13 Entrada (Botón)

    while(1) {
        // Leemos el registro IDR del Puerto C, y filtramos el bit 13
        // Si el resultado es distinto de cero, el botón NO está presionado
        if (GPIOC->IDR & (1 << 13)) {
            // Botón suelto (Lee un 1 lógico)
            // Apagamos el LED usando AND y NOT
            GPIOA->ODR &= ~(1 << 5);
        } else {
            // Botón presionado (Lee un 0 lógico)
            // Encendemos el LED usando OR
            GPIOA->ODR |= (1 << 5);
        }
    }
}
```

Este método de meter la pregunta `if` dentro del bucle infinito `while(1)` se llama **Polling (Encuesta)**. Funciona, pero desperdicia mucha energía de la CPU preguntando un millón de veces por segundo.

13. Interrupciones Externas (EXTI): Trabajar Inteligente

En ingeniería seria, no usamos Polling para leer botones o recibir alarmas. Usamos **Interrupciones**. Consiste en configurar el hardware para que el microcontrolador se vaya a dormir o se dedique a cálculos complejos, y que el hardware físico le "grite" (interrumpa) solo en el momento exacto en que detecta que se pulsó el botón.

Para esto, la familia STM32 cuenta con el **EXTI (Extended Interrupts and Events Controller)**, un vigilante de hardware independiente.

> **Líneas EXTI (Las cámaras de seguridad)**
> El vigilante EXTI no vigila pin por pin, vigila por "Líneas". Todos los pines número 13 (PA13, PB13, PC13) están conectados internamente a la Línea 13 (EXTI13). Tenemos que decirle a un multiplexor (registro SYSCFG) que conecte específicamente el Puerto C a la Línea 13.

Registros Clave de EXTI

- `EXTI->IMR1` (Interrupt Mask Register): Enciende la alarma. Si ponemos un 1 en el bit 13, "desenmascaramos" (activamos) la alarma para la Línea 13.
- `EXTI->FTSR1` (Falling Trigger Selection): Define qué detecta la alarma. Si le ponemos un 1 al bit 13, la alarma sonará por un "flanco de bajada" (cuando el voltaje cae de 3.3V a 0V, es decir, cuando se presiona el botón).
- `EXTI->PR1` (Pending Register): Cuando la alarma suena, este bit se pone en 1 automáticamente. Tenemos que borrarlo manualmente (escribiéndole un 1, por raro que suene, en el STM32 se borra escribiendo 1) para indicarle al vigilante que ya fuimos a revisar el problema.

14. Configurar EXTI e ISR (Interrupt Service Routine)

Además del vigilante (EXTI), necesitamos despertar al Gerente Central del procesador ARM: el **NVIC (Nested Vectored Interrupt Controller)**, quien es el encargado de pausar el código principal y saltar a la función de emergencia.

Código Completo: Interrupción al presionar PC13

```
void EXTI_Init(void) {
    // 1. Habilitar reloj para el configurador de sistema (SYSCFG)
    RCC->APB2ENR |= RCC_APB2ENR_SYSCFGEN;

    // 2. Enrutar el PC13 a la línea EXTI13. (Registro EXTICR4, bits 7:4 a valor 0010
para Puerto C)
    SYSCFG->EXTICR[3] &= ~(0xF << 4); // Limpiar
    SYSCFG->EXTICR[3] |=  (0x2 << 4); // Seleccionar Puerto C (0x2)

    // 3. Configurar EXTI para disparar en flanco de bajada (Falling edge - presionar)
    EXTI->FTSR1 |= (1 << 13);

    // 4. Desemmascarar (activar) la línea 13
    EXTI->IMR1 |= (1 << 13);

    // 5. Activar la interrupción en el gerente principal de la CPU (NVIC)
    NVIC_EnableIRQ(EXTI15_10_IRQn); // La línea 13 está en el grupo de líneas 10 al 15
}

// ESTA ES LA RUTINA DE EMERGENCIA (El nombre debe ser EXACTAMENTE este, viene de ARM)
void EXTI15_10_IRQHandler(void) {
    // Revisar si realmente la alarma vino de la Línea 13
    if (EXTI->PR1 & (1 << 13)) {

        GPIOA->ODR ^= (1 << 5); // Alternamos el LED PA5 (Acción rápida)

        // MUY IMPORTANTE: Limpiar la alarma (Registro Pending) escribiendo un 1
        // Si no hacemos esto, la CPU se quedará trabada eternamente aquí.
        EXTI->PR1 |= (1 << 13);
    }
}
```

15. Timers (Cronómetros de Hardware)

Usar bucles `while(contador--)` como hicimos antes para generar retrasos es terrible. Depende de la temperatura del chip, de otras interrupciones, y congela la placa por completo. El STM32 incluye múltiples **Timers (Temporizadores)**. Son pequeños bloques de hardware que cuentan pulsos de reloj (hacen Tic-Tac) de manera totalmente independiente en segundo plano.

Registros Clave del Timer Básico (Ej. TIM6)

El TIM6 está conectado a un reloj interno de la CPU (por ejemplo, a 170 MHz).

- `TIM6->PSC` (Prescaler Register): Divide el reloj principal. Si a 170MHz le pones un prescaler de 170-1, el timer hará un "Tic" a exactamente 1 MHz (1 millón de tics por segundo, 1 tic = 1 microsegundo).
- `TIM6->ARR` (Auto-Reload Register): Es el "techo" de la cuenta regresiva. Si el Timer hace tic cada 1 us, y pones un ARR de 1000, el Timer contará hasta 1000 us (1 milisegundo) y luego volverá a 0 lanzando una bandera de aviso (Update Event).
- `TIM6->CR1` (Control Register 1): Su bit número 0 (CEN - Counter Enable) sirve para encender y arrancar el cronómetro.

Configurar Timer para retardos de 1 milisegundo

```
void Timer6_Init(void) {
    // 1. Dar energía al TIM6 (Bus APB1)
    RCC->APB1ENR1 |= RCC_APB1ENR1_TIM6EN;

    // 2. Configurar Prescaler.
    // Suponiendo reloj a 16MHz (reset default): PSC = 16-1 = 15.
    // Ahora hace 1 tic cada 1 microsegundo.
    TIM6->PSC = 16 - 1;

    // 3. Configurar Techo (Auto-Reload). 1000 tics = 1 milisegundo.
    TIM6->ARR = 1000 - 1;

    // 4. Arrancar el Timer (Bit CEN = 1)
    TIM6->CR1 |= TIM_CR1_CEN;
}
```

16. Crear una función de retardo perfecta (Delay)

Usando el Timer 6 que configuramos para que "reinicie" su cuenta (y levante una bandera) exactamente cada 1 milisegundo, podemos crear una función `delay_ms(tiempo)` profesional, precisa y que no depende de la velocidad del compilador.

```
// Función de retardo bloqueante usando hardware (Polling del Timer)
void delay_ms(uint32_t ms) {
    for (uint32_t i = 0; i < ms; i++) {
        // Limpiar la bandera de aviso antes de contar
        // (Registro de estado SR, bit 0 Update Interrupt Flag - UIF)
        TIM6->SR &= ~TIM_SR_UIF;

        // Reiniciar el contador físico interno del timer a 0
        TIM6->CNT = 0;

        // Esperar atrapados en un bucle vacío HASTA que el hardware
        // levante la bandera UIF, lo que indica que pasaron los 1000 tics (1ms)
        while (!(TIM6->SR & TIM_SR_UIF)) {
            // Esperar...
        }
    }
}

int main(void) {
    System_Clock_Init();
    GPIO_Init();
    Timer6_Init();

    while(1) {
        GPIOA->ODR ^= (1 << 5); // Alternar LED
        delay_ms(500);          // Retraso perfecto de medio segundo
    }
}
```

> **El Siguiente Nivel**
>
> Aunque esto es muy preciso, sigue atrapando a la CPU en el `while`. El verdadero objetivo es conectar el `TIM_SR_UIF` directamente a una interrupción **NVIC**, de manera que la CPU siga haciendo matemática compleja y el Timer interrumpa al código principal 1 vez cada milisegundo (la base de cualquier Sistema Operativo en Tiempo Real como FreeRTOS).

17. PWM (Modulación por Ancho de Pulso)

Los microcontroladores son digitales: solo sacan 3.3V (Encendido) o 0V (Apagado). ¿Cómo logras entonces que un motor gire a la mitad de su velocidad, o que un LED brille a media intensidad? Cambiando el pin de 1 a 0 miles de veces por segundo. Si está el 50% del tiempo en Alto y 50% en Bajo, físicamente promediará ~1.65V. A esto se le llama Duty Cycle (Ciclo de trabajo).

En STM32, los Timers avanzados (TIM1, TIM2, TIM3, etc.) tienen sub-bloques dedicados a sacar estas ondas perfectas por los pines físicos **automáticamente y por hardware**, sin gastar 1 solo ciclo de procesamiento en la CPU.

El Registro Mágico: `CCR1` (Capture/Compare Register)

En el Modo PWM, el Timer cuenta desde 0 hasta el Techo (ARR). Puedes decirle al hardware: "Saca voltaje 3.3V al pin, y cuando el contador interno cruce el valor que yo guardé en CCR1, corta el voltaje a 0V hasta que vuelvas a iniciar".

- Si ARR = 1000 y pones CCR1 = 500 : Ciclo del 50% (Media energía).
- Si ARR = 1000 y pones CCR1 = 100 : Ciclo del 10% (Poca energía).

Configurando PWM en PA5 (TIM2 Canal 1)

```
void PWM_Init(void) {
    // 1. El Pin PA5 debe estar en "Modo Alterno" (10), NO en Salida Normal (01).
    GPIOA->MODER &= ~(3 << 10);
    GPIOA->MODER |=  (2 << 10);
    GPIOA->AFR[0] |= (1 << 20); // Función Alterna 1 (Asigna PA5 al TIM2_CH1)

    // 2. Habilitar Energía al TIM2
    RCC->APB1ENR1 |= RCC_APB1ENR1_TIM2EN;

    // 3. Configurar periodo (Frecuencia del PWM). Asumimos ARR = 100
    TIM2->PSC = 16 - 1;
    TIM2->ARR = 100 - 1; // Frecuencia de 10 kHz

    // 4. Configurar el Canal 1 en Modo PWM 1 (Registro CCMR1, bits OC1M a 0110)
    TIM2->CCMR1 |= (6 << 4); // Modo PWM 1
    TIM2->CCMR1 |= TIM_CCMR1_OC1PE; // Habilitar precarga

    // 5. Activar la salida física del canal (CCER)
    TIM2->CCER |= TIM_CCER_CC1E;

    // 6. Arrancar el Timer
    TIM2->CR1 |= TIM_CR1_CEN;
}

// Para cambiar el brillo en cualquier parte del código:
// TIM2->CCR1 = 20; // Brillo al 20%
```

18. El Mundo Analógico (ADC)

Los sensores de temperatura (PT100, termopares amplificados), celdas de carga para peso, o incluso un simple potenciómetro, no entregan unos y ceros. Entregan un voltaje que varía suavemente (por ejemplo, 1.45 Voltios).

El procesador digital no entiende "1.45 Voltios". Necesita el hardware interno llamado **ADC (Analog-to-Digital Converter)**. El ADC tomará una "foto" del voltaje y lo traducirá a un número entero que C pueda procesar.

> **Resolución del ADC (12 Bits)**
>
> El STM32G4 tiene potentes conversores de 12 bits de ultra alta velocidad. Al ser de 12 bits, el número digital máximo es (2 elevado a la 12) - 1 = **4095**.
>
> - Si en el pin hay 0.0 Voltios (GND) -> El ADC leerá el número 0.
> - Si en el pin hay 3.3 Voltios (VCC) -> El ADC leerá el número 4095.
> - Si en el pin hay 1.65 Voltios (Mitad) -> El ADC leerá el número 2047.

Registros Clave del ADC (ADC1)

- `ADC1->CR` (Control Register): Bit `ADEN` lo enciende, bit `ADSTART` da la orden de disparar una conversión instantánea y leer.
- `ADC1->SQR1` (Sequence Register): Los ADCs tienen multiplexores (muchos canales entran al mismo hardware). Aquí defines la longitud de la lista de canales a leer (L) y cuál es el primer canal (SQ1).
- `ADC1->DR` (Data Register): **El Premio.** Cuando el ADC termina su trabajo matemático, deposita el número resultante (0-4095) en los primeros 16 bits de este registro.

Nota importante sobre la familia STM32G4: A diferencia de series antiguas (F1/F4), la arquitectura G4 es orientada a procesamiento analógico de precisión. Antes de encender el ADC, exige calibrar su hardware interno y usar registros avanzados para estabilizar su voltaje (ADC Voltage Regulator).

19. Leer un Sensor Analógico (Código)

Usaremos el pin PA0, que corresponde internamente al ADC1_IN1 (Canal 1 del Convertidor Analógico 1).

```
void ADC1_Init(void) {
    // 1. Pin PA0 en Modo Analógico (Bits 0 y 1 en 11)
    RCC->AHB2ENR |= RCC_AHB2ENR_GPIOAEN;
    GPIOA->MODER |= (3 << 0);

    // 2. Habilitar reloj para el bloque de ADCs (AHB2)
    RCC->AHB2ENR |= RCC_AHB2ENR_ADC12EN;

    // 3. Configurar reloj interno del ADC a sincrónico
    ADC12_COMMON->CCR |= (1 << 16);

    // 4. Sacar el ADC de "Deep Power Down" y habilitar su regulador
    ADC1->CR &= ~ADC_CR_DEEPPWD;
    ADC1->CR |= ADC_CR_ADVREGEN;
    delay_ms(1); // Esperar a que el hardware de voltaje despierte

    // 5. Encender el ADC (ADEN)
    ADC1->CR |= ADC_CR_ADEN;
    while(!(ADC1->ISR & ADC_ISR_ADRDY)); // Esperar bandera de "Ready"

    // 6. Configurar el Secuenciador: Solo 1 conversión, leyendo el Canal 1
    ADC1->SQR1 = (1 << 6); // SQ1 = Canal 1
}

uint16_t Leer_Voltaje(void) {
    // Disparar conversión de software
    ADC1->CR |= ADC_CR_ADSTART;

    // Esperar (bucle cerrado) hasta que termine la conversión
    // Bandera EOC (End of Conversion)
    while (!(ADC1->ISR & ADC_ISR_EOC));

    // Retornar el número (0-4095) almacenado en el Data Register
    return ADC1->DR;
}
```

20. Comunicación (UART) y Siguientes Pasos

Ya puedes mover motores y leer sensores, pero necesitas enviar y recibir datos en texto a tu computadora (o a un módulo Bluetooth ESP32). Para esto se utiliza el **UART (Universal Asynchronous Receiver-Transmitter)**. Es la clásica comunicación serial de 2 cables cruzados: Transmisor (TX) y Receptor (RX).

Registros Clave (USART2)

El USART2 de la placa Nucleo está conectado físicamente por las pistas de cobre de la placa directa al cable USB, lo que te permite comunicarte con la PC sin cables extra.

- `USART2->BRR` (Baud Rate Register): Configura la velocidad. Si tu reloj va a 16MHz y quieres velocidad típica de 115200 baudios, divides (16000000 / 115200) = 138 (0x8A) y lo guardas aquí.
- `USART2->TDR` (Transmit Data Register): Para enviar la letra 'A', escribes `USART2->TDR = 'A';`. El hardware serializará los 8 bits de la 'A' y los sacará 1 por 1 por el cable físico.

Enviar Texto (El fin del inicio)

```
// Función simple para enviar un solo caracter a la PC
void UART2_Enviar_Caracter(char c) {
    // Enviar el caracter al registro transmisor
    USART2->TDR = c;

    // Esperar obligatoriamente hasta que la transmisión física acabe
    // Bit TXE (Transmit Data Register Empty) en el registro de Status (ISR)
    while (!(USART2->ISR & USART_ISR_TXE_TXFNF));
}

// Y si quieres enviar toda una palabra (un arreglo de caracteres terminados en Null):
void UART2_Enviar_String(char *cadena) {
    while (*cadena) {
        UART2_Enviar_Caracter(*cadena++);
    }
}
// Uso: UART2_Enviar_String("Hola desde STM32 Bare Metal!\n");
```

El Próximo Horizonte: DMA y Debugging

Todo el código anterior usa el procesador (CPU) para mover la información de un registro a otro. A medida que escales, aprenderás a usar el DMA (Direct Memory Access), un circuito capaz de tomar todas las conversiones de tu ADC y guardarlas en tu Memoria RAM sin que tu código principal `main()` tenga que ejecutar una sola línea de código.

Usa STM32CubeMonitor con tus conocimientos de la variable `volatile` para ver en la pantalla de tu computadora los valores físicos subiendo y bajando en gráficas reales. ¡Bienvenido a la ingeniería embebida!